



- **Frontend (public/):** Es la interfaz gráfica que ve y toca el usuario en su navegador. No tiene acceso directo a bases de datos ni archivos del sistema por seguridad. Su trabajo es recolectar datos e invocar al servidor vía llamadas HTTP.
- **Backend (src/):** Es el motor invisible que corre en el servidor. Su trabajo es recibir peticiones HTTP, ejecutar lógica de negocio, validar datos, proteger contraseñas y responder con datos estructurados en formato JSON.

BACKEND?

Se comunican a través del protocolo **HTTP** utilizando el formato de texto ligero **JSON**.

El viaje de los datos en código:

1. El Frontend lanza la llamada (`public/app.js`):

```
// Petición asíncrona con fetch() (Devuelve una Promise)
const respuesta = await fetch('http://localhost:3000/');
const productos = await respuesta.json(); // Traduce
```

2. El Backend abre la puerta (`src/server.ts`):

- `this.app.use(cors())`: Permiso de seguridad para aceptar llamadas del navegador.
- `this.app.use(express.json())`: Traduce el texto JSON a objeto usable en `req.body`.

3. El Controlador procesa y responde (`src/controllers/products.controller.ts`):

```
getProducts(_req: Request, res: Response) {
  res.json(products); // Envía los datos por el cable
}
```

3. EXPLICACIÓN DETALLADA DE `req` Y `res` (Parámetros del Controlador)

En Express, cada función o método de un controlador recibe dos parámetros fundamentales inyectados automáticamente por el framework:

```
metodo(req: Request, res: Response) { ... }
```

A) **req** = Request (La Petición Entrante)

Es un objeto gigante que representa todo lo que el cliente envió hacia el servidor:

- **req.body**: El cuerpo de la petición. Contiene los datos enviados en formularios o envíos POST/PUT (ej. `{ nombre: "Sabritas", precio: 18.50 }`).
- **req.params**: Los parámetros dinámicos definidos en la URL (ej. En `/api/v1/products/:id`, `req.params.id` obtiene `"2"`).
- **req.query**: Parámetros de búsqueda en la URL después del signo `?` (ej. En `/search?color=azul`, `req.query.color` obtiene `"azul"`).

B) **res** = Response (La Respuesta Saliente)

Es la herramienta que usa el servidor para responderle al cliente y cerrar la conexión socket:

- **res.json(datos)**: Convierte un objeto o arreglo a JSON, le pone la cabecera `Content-Type: application/json` y lo envía al cliente.
- **res.status(codigo)**: Define el código de estado HTTP (200 = OK, 201 = Creado, 400 = Error cliente, 404 = No encontrado, 500 = Error servidor).
- **res.send("texto")**: Envía texto plano o HTML básico.

4. CÓDIGO ESPECÍFICO DE CADA ARCHIVO BACKEND

A) Archivo `.env` (Configuración de secretos)

PORT=3000

B) Archivo `src/controllers/products.controller.ts` (Lógica de negocio)

```
import type { Request, Response } from "express";

const products = [
  { id: 1, name: "Galletas Marias 170g", price: 16.50 },
  { id: 2, name: "Refresco Coca Cola 600ml", price: 19.00 }
];

export class ProductController {
  // GET /api/v1/products
  getProducts(_req: Request, res: Response) {
    res.json(products);
  }

  // POST /api/v1/products
  createProduct(req: Request, res: Response) {
    const { name, price } = req.body;
    if (!name || !price) {
      return res.status(400).json({ message: "El nombre y"
    }
    const product = { id: products.length + 1, name, price };
    products.push(product);
    res.status(201).json({ message: "ok", product });
  }

  // PUT /api/v1/products/:id
  updateProduct(req: Request, res: Response) {
    const id = +req.params.id!; // '+' convierte a número
    const { name } = req.body;
    const product = products.find(p => p.id === id);

    if (!product) {
      res.status(404).json({ message: "product not found"
    }
  }
}
```

```

        return;
    }
    product.name = name;
    res.status(200).json({ message: "product updated" });
}
}

```

C) Archivo `src/controllers/user.controller.ts` (Ocultar contraseñas)

```

import type { Request, Response } from "express";

const users = [
  { id: 1, name: "Leonardo", role: "ADMIN", password: "Secret" },
];

export class UserController {
  getUserById(req: Request, res: Response) {
    const id = Number(req.params.id);
    const user = users.find(u => u.id === id);

    if (!user) {
      res.status(404).json({ message: "user not found" });
      return;
    }

    // Operador Spread {...user} para copiar datos y ocultar contraseña
    res.json({ ...user, password: "" });
  }
}

```

D) Archivo `src/routes/products.routes.ts` (Conectar Verbos HTTP)

```

import { Router } from "express";
import { ProductController } from "../controllers/product.controller";

```

```
const productController = new ProductController();
const router = Router();

router.get('/', (req, res) => productController.getProduct);
router.post('/', (req, res) => productController.createProduct);
router.put('/:id', (req, res) => productController.updateProduct);

export default router;
```

E) Archivo `src/routes/index.ts` (Router Maestro con Versionamiento)

```
import { Router } from "express";
import productsRoutes from './products.routes.ts';
import userRoutes from './user.routes.ts';

const router = Router();

router.use('/v1/products', productsRoutes);
router.use('/v1/users', userRoutes);

export default router;
```

F) Archivo `src/server.ts` (Clase Server POO)

```
import express, { Express, Router } from "express";
import cors from "cors";

type ServerOptions = {
  port: number;
  routes: Router;
};

export class Server {
  private readonly port: number;
```

```

private readonly app: Express;
private readonly routes: Router;

constructor(options: ServerOptions) {
  this.app = express();
  this.port = options.port;
  this.routes = options.routes;
}

public start = (): void => {
  this.app.use(cors());
  this.app.use(express.json());
  this.app.use('/api', this.routes);

  this.app.listen(this.port, () => {
    console.log(`🚀 Servidor TypeScript en http://localhost:${this.port}`);
  });
};
}

```

G) Archivo `src/app.ts` (Punto de Entrada)

```

import { Server } from "../server.ts";
import routes from "../routes/index.ts";
import dotenv from "dotenv";

dotenv.config();

const port = Number(process.env.PORT) || 3000;

function main(): void {
  const server = new Server({ port, routes });
  server.start();
}

main();

```

5. JAVASCRIPT vs TYPESCRIPT (Diferencias Teóricas y de Código)

Concepto	JavaScript (JS)	TypeScript (TS)
Definición	Lenguaje original de tipado dinámico.	Superconjunto de JS con tipado estático (como C++/Java) .
Errores	Descubres errores al ejecutar el código en vivo.	VS Code detecta errores mientras estás escribiendo el código.
Tipado de variables	<code>let puerto = 3000;</code>	<code>let puerto: number = 3000;</code>
Firma de Funciones	<code>getProducts(req, res)</code>	<code>getProducts(req: Request, res: Response): void</code>
POO	<pre>class Server { constructor(port) { this.port = port; } }</pre>	<pre>class Server { private readonly port: number; ... }</pre>
Ejecución	Se ejecuta directo en Node/Navegador.	Se ejecuta mediante <code>tsx</code> (<code>npm run tsx --watch ./src/app.ts</code>).

6. ROADMAP DEFINITIVO DESDE 0 (Para crear cualquier proyecto nuevo)

PASO 1: Terminal e Inicialización de Paquetes

```
npm init -y
```

```
npm install express cors dotenv
npm install typescript @types/node @types/express @types/
```

🚩 PASO 2: Configurar Archivos Base

1. `.env`: `PORT=3000`
2. `tsconfig.json`:

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "esModuleInterop": true,
    "strict": true,
    "skipLibCheck": true,
    "allowImportingTsExtensions": true,
    "noEmit": true
  }
}
```

3. `package.json`:
 - Poner `"type": "module"`
 - En `"scripts"`: `"dev": "npx tsx --watch ./src/app.ts"`

🚩 PASO 3: Construir el Backend en `src/` (En este orden)

1. `src/controllers/` (`products.controller.ts`, `user.controller.ts`)
2. `src/routes/` (`products.routes.ts`, `user.routes.ts`, `index.ts` con `/v1`)
3. `src/server.ts` (Clase `Server` encapsulada)
4. `src/app.ts` (Ejecutable `main()`)

🚩 PASO 4: Encender el Servidor Backend (Comando de Terminal)

En la terminal de la carpeta del proyecto, ejecuta:

```
npm run dev
```

(Verás en consola: 🚀 Servidor TypeScript ejecutándose en `http://localhost:3000`)

🚩 PASO 5: Construir el Frontend en `public/`

1. `public/index.html` (Formularios, inputs con `id="..."`, divs de contenedor)
2. `public/style.css` (Estilos y layout `grid/flex`)
3. `public/app.js` (`document.getElementById`, `addEventListener`, `fetch()` con `await`)